

Laboratorio 8

Redes Neuronales Convolucionales

(con Keras)

Notas

Valentina Pérez Cadavid

2026-2

Ahora se mirará la red como una imagen 2D, se usan filtros convolucionales

Una red es una función matemática que transforma una entrada en una salida, pasando la información por varias capas de operaciones (productos, sumas, no linealidades como ReLU (función de activación) o sigmoide)

Cada una de estas operaciones tienen parámetros ajustables (los pesos). Entrenar la red es ir ajustando cada uno de los pesos hasta que la salida se aproxime a un valor esperado

En una red neuronal convolucional, un MLP (multilayer perceptron) toma la imagen, la aplanar en una lista de números, y cada neurona de la primera capa se conecta con todos los píxeles de la imagen. Aquí surgen 2 problemas:

- 1) Se pierde la información de qué píxel está al lado de cual
- 2) El número de conexiones (pesos) es muy grande si la imagen es grande

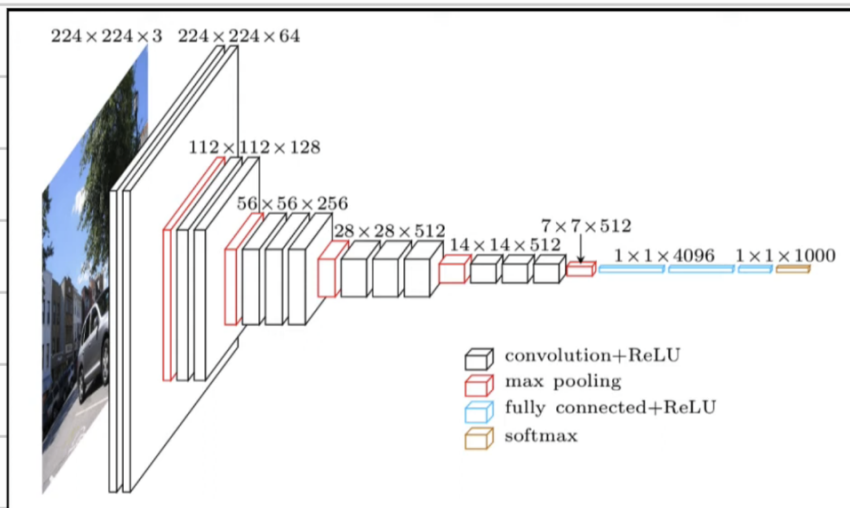
Una red convolucional resuelve estos problemas usando un filtro (Kernel) que se desliza sobre la imagen. Estos filtros se especializan en extraer diferentes características de la imagen.

En cada posición, el filtro hace un producto punto local con el pedazo de imagen y ese resultado va formando una nueva imagen llamada mapa de activación

El filtro no se diseña, la red lo aprende

La salida de estos filtros, se llevan a unas capas llamadas **max pooling**, las cuales permiten reducir la cantidad de datos y extraer los datos mas representativos de los filtros.

Esto se hace de forma sucesiva y por tanto el tamaño de las imágenes es cada vez más pequeño



En la etapa final de la red, se toman todas las características que obtuvieron las capas convolucionales. Estas características se representan como un vector de datos y se llevan a una capa **softmax** (ó sigmoidal) que permite finalmente realizar la clasificación de la imagen.

12	1	12	2	2	9
2	3	1	3	5	7
1	5	6	7	8	9
1	3	34	12	11	10
-1	3	6	7	9	0
2	3	8	8	1	2

 $\times$

1	0	-1
1	0	-1
1	0	-1

 $=$

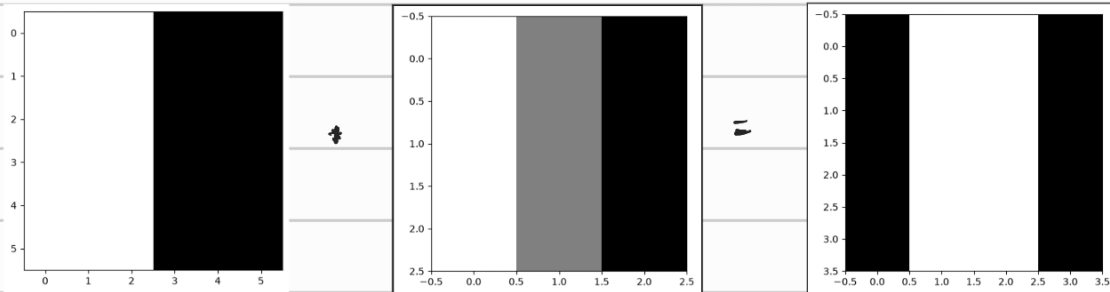
-4			

$$\begin{aligned}
 &\cancel{12 \cdot 1} + 2 \cdot 1 + \cancel{1 \cdot 1} + \\
 &1 \cdot 0 + 3 \cdot 0 + 5 \cdot 0 \\
 &12 \cdot (-1) + 1 \cdot (-1) + 6 \cdot (-1) = -4 \quad \dots
 \end{aligned}$$

Dada una imágen M , se construye una operación de convolución a partir de un filtro

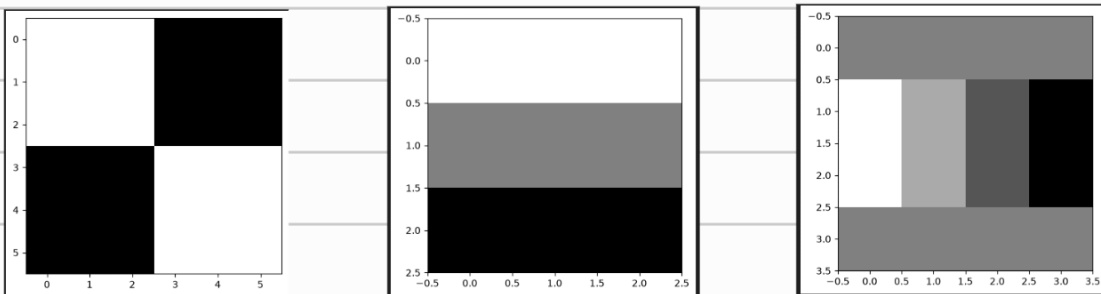
$$\text{Resultado} = M * \text{Filter}$$

Filtro vertical:



$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Filtro horizontal:



$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Sobel filter:
$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

Scharr Filter:
$$\begin{bmatrix} 3 & 0 & -3 \\ 10 & 0 & -10 \\ 3 & 0 & -3 \end{bmatrix}$$

→ Si se quiere detectar un cambio (un borde), los pesos deben sumar 0: $[1 \ 0 \ -1] \rightarrow 0$

→ El patrón de signos define en qué dirección aplica el filtro

- positivo a la izquierda, negativo derecha → detecta cambios horizontales
- positivo arriba, negativo abajo → detecta cambios verticales

→ Si se quiere detectar un promedio por zona y no cambios, los pesos deben ser positivos

Cómo elegir los parámetros del filtro para detectar otro tipo de bordes?

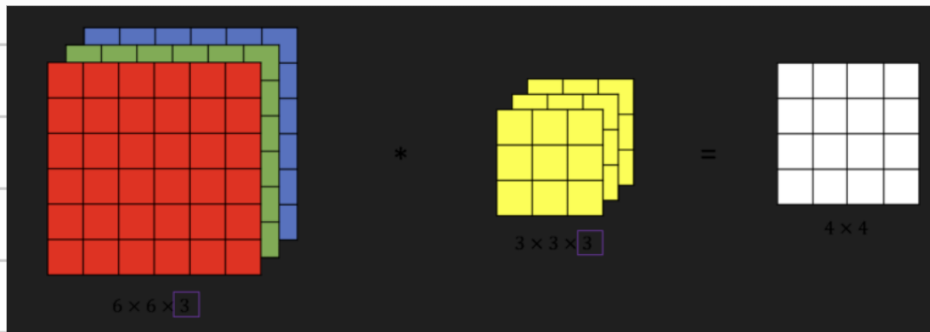
$$\begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix}$$

Padding: permite que el tamaño de la imagen resultante sea del mismo tamaño de la imagen original

$$\begin{matrix} \text{imagen} & \text{filtro} & \rightarrow & \text{output} \\ (n \times n) & (f \times f) & & (n-f+1) \times (n-f+1) \end{matrix}$$

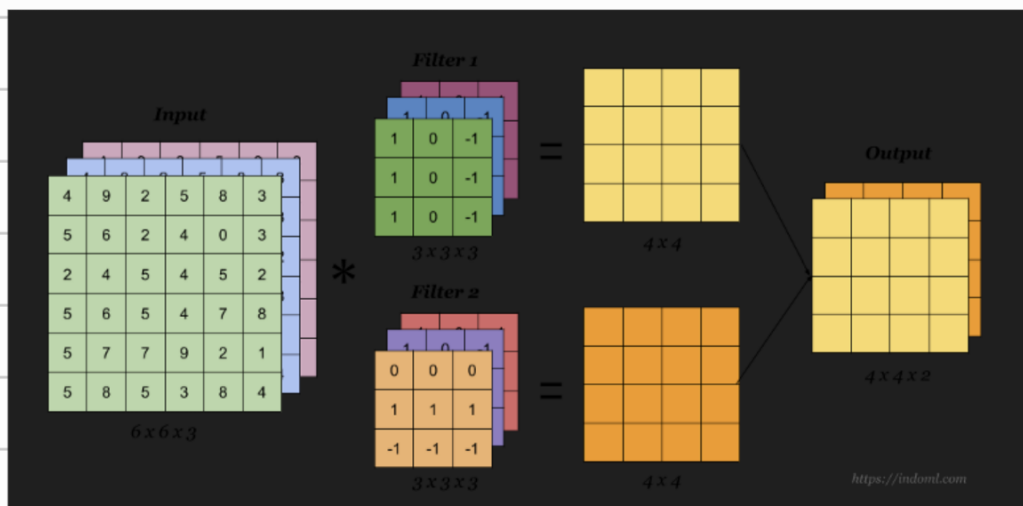
Stride: indica cuántos pasos salta la ventana cada vez que se desliza, en vez de moverse de a un pixel/columna a la vez

Cuando se trabajan con imágenes, se trabaja con 3 canales RGB. cada uno de estos canales tiene su propia matriz del tamaño de la imagen. En el caso de una imagen 6×6 , se consigue entonces un volumen $6 \times 6 \times 3$.



Por tanto, para convolucionar sobre un volumen, el filtro también debe tener profundidad. Una capa 3×3 distinta para cada canal.

Al hacer la convolución, se toma la porción 3×3 de los 3 canales a la vez (27 números), se multiplican elemento por elemento con los 27 pesos del filtro y se suma todo en un solo número. El resultado es plano.



Esto mismo se aplica cuando hay más de 1 filtro.

Cada filtro se aplica por separado, de forma completamente independiente. Como resultado de la convolución, cada filtro produce su propio mapa (4×4).

Al final, cada mapa se aplana formando una salida $4 \times 4 \times 2$.

$$\text{output} = \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times k$$

n : tamaño de la imagen de entrada (asumiendo cuadrada)
 f : tamaño del filtro (asumiendo cuadrada)
 p : padding (relleno agregado en cada borde)
 s : stride (salto de la ventana)
 k : # de filtros

Clasificación de fases del modelo de Ising con una red convolucional (Keras)

- Para esto:
1. Se cargarán los datos ya generados
 2. Armar y entrenar la CNN
 3. Usar la red entrenada para estimar T_c
 4. Usar la red, sin reentrenar, sobre configuraciones de otro tamaño $L=20$

Keras: API para construir y entrenar redes neuronales. No calcula nada por sí mismo, sino que internamente usa TensorFlow, PyTorch, JAX

El backend se elige con la variable de entorno `KERAS_BACKEND`:

Keras contiene el ciclo de entrenamiento

PyTorch (manual)	Keras (equivalente)
Definir Dataset + DataLoader	Pasar arreglos de NumPy directamente a <code>.fit()</code>
Escribir el bucle <code>for epoch ... for batch ...</code>	<code>model.fit(X, y, epochs=..., batch_size=...)</code>
<code>loss.backward()</code> , <code>optimizer.step()</code> , <code>optimizer.zero_grad()</code>	Ocurre internamente en <code>.fit()</code>
Guardar el mejor modelo con un <code>if val_loss < best_val_loss</code>	Callback <code>keras.callbacks.ModelCheckpoint</code>
<code>model.eval()</code> + <code>torch.no_grad()</code> + bucle manual	<code>model.evaluate(X, y) / model.predict(X)</code>

→ calcular el gradiente del error respecto a los pesos, actualizar pesos

→ epoch / época: significa cuantas veces pasa el conjunto de entrenamiento completo por la red en total, de principio a fin.

→ Batch / lote: subgrupo de ejemplos que se procesan juntos antes de actualizar los pesos

1. Datos: configuraciones de espines del modelo de Ising

fase ordenada ($y=0$)

$$T < T_c$$

Dominio grande

fase desordenada ($y=1$)

$$T > T_c$$

Sin dominios

$x = (1600, 16, 16)$

$y = (1600,)$

$T = [T_1, \dots, T_{10}]$

realizaciones = $[0, 1, 2, 3]$

$L = 16$

• Obtener 2 configuraciones de ejemplo y graficarlas

→ $y == 0$ Da arreglo de True/False (¿esta posición fase ordenada?)

→ $\text{np.where}(y == 0)$: Devuelve los índices donde es True, en una tupla

→ $\text{np.where}(y == 0)[0][0]$:

$y = [1 \ 1 \ 0 \ 1 \ 0]$

→ $y == 0 : [F \ F \ T \ F \ T]$

→ $\text{np.where}(y == 0) : (\text{array}([2, 4]),)$

→ $\text{np.where}(y == 0)[0] : \text{array}([2, 4])$

→ $\text{np.where}(y == 0)[0][0] = 2$ → la posición 2 del data set es la primera configuración ordenada

2. Preparación de los datos para Keras

Realización : 0 y 1 → Entrenamiento

Realización : 2 → validación

Realización : 3 → Test

En lugar de aplanar, se necesita conservar la forma 2D, mas una tercera dimensión para los canales. Dado que solo hay un único canal (valor del espín) la forma sería $(16 \times 16 \times 1)$

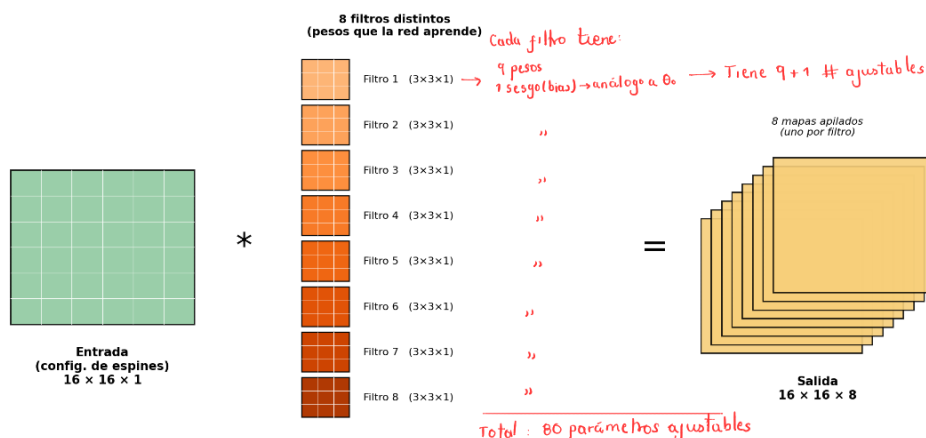
3. Arquitectura : red convolucional simple

Los filtros convolucionales detectan patrones locales (espines alineados con sus vecinos) sin importar la posición.

- `Conv2D(8, 3x3) + ReLU` — detecta patrones locales de alineación.
- `MaxPooling2D(2x2)` — reduce la resolución espacial.
- `Conv2D(16, 3x3) + ReLU` — combina patrones locales en patrones un poco más grandes.
- `GlobalAveragePooling2D` — en vez de aplanar (Flatten) y usar una capa densa grande, promediamos cada mapa de activación sobre todo el espacio. Esto produce un vector de tamaño fijo **sin importar el tamaño $L \times L$ de la entrada**: la misma red podrá evaluarse más adelante sobre configuraciones de otro tamaño de red, algo que el MLP (con su entrada aplanada de tamaño fijo) no puede hacer.
- `Dense(16) + ReLU` y `Dense(1) + sigmoide` — combinan las características globales en una probabilidad de fase desordenada.

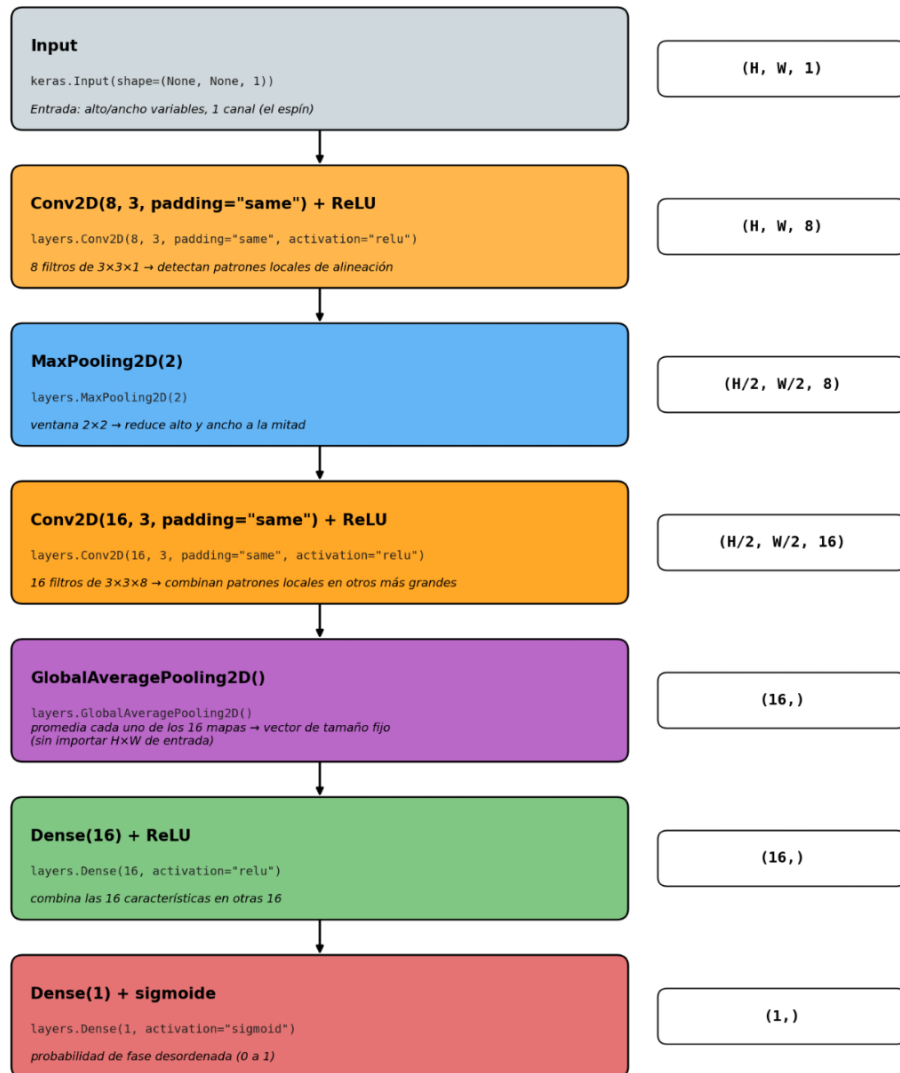
→ (16, 16, 8)
→ (8, 8, 8)
→ (8, 8, 16)

Primera capa: `Conv2D(8, 3x3, padding="same")` sobre una configuración de Ising



Arquitectura completa: `model = keras.Sequential(..., name="ising_cnn")`

forma de salida
(ejemplo con L=16)



Recibe 16 números de la capa anterior y tiene 16 neuronas de salida.

Cada una de estas 16 neuronas necesita un peso por cada uno de los 16 números que recibe más su propio sesgo:

Neurona 1: 16 pesos + 1 sesgo = 17

Neurona 2: 16 pesos + 1 sesgo = 17

⋮

Neurona 16: 16 pesos + 1 sesgo = 17

Total = 272 parámetros ajustables

4. Compilación y Entrenamiento

Dado que la última capa ya tiene "sigmoid", entonces "model.predict()" devuelve directamente una probabilidad entre 0 y 1

"binary_crossentropy" está diseñada para comparar exactamente la probabilidad predicha contra la etiqueta real. (0 ó 1)

→ ms/step : tiempo que demora en procesar un solo batch

→ accuracy : % de aciertos

→ loss : indicador de qué tan mal predijo la red.
loss bajo = buena predicción

} se obtienen de promediar sobre los 25 batches de cada epoch.

→ val-accuracy } se mide una sola vez al final ya con los pesos actualizados
→ val-loss } por los 25 batches completados de una epoch en específico.

5. Evaluación en el conjunto de Test

model.predict() : recordar que devuelve el arreglo de tamaño (N,1) que contiene las probabilidades de fase desordenada.

Este arreglo luego se convierte en un arreglo de 1 y 0 :

1 → si prob $>$ 0.5 desordenada.

0 → si prob $<$ 0.5 ordenada.

→ resumen [] : filtra filas

→ resumen [" "] : filtra columnas

7. Bonus: Generalization a otro tamaño de red (L=20)

GlobalAveragePooling2D hace que la red no tenga ningún peso atado a un tamaño de entrada específico. Se puede tomar el mismo modelo, sin reentrenar, y evaluarlo sobre temperatura_critica20.npy.

Si usamos MLP (flatten+Dense) el número de pesos de la primera capa está fijo por el tamaño de entrada.

En el caso de Conv2D, los filtros tienen siempre el mismo número de pesos sin importar el tamaño de la imagen.

Es decir, la red completa nunca memoriza el tamaño 16×16 , solo aprendió patrones locales (a través de los filtros) y un resumen global, ambos independientes del tamaño de la imagen.

Ejm: $N=2$ configuraciones

$$\text{datos_tc3}["x"] = \begin{bmatrix} 1 & 1 & -1 \\ 1 & -1 & -1 \\ -1 & 1 & 1 \end{bmatrix}, \begin{bmatrix} -1 & -1 & 1 \\ -1 & 1 & 1 \\ 1 & 1 & -1 \end{bmatrix} \rightarrow \text{datos_tc3}["x"] \quad (2, 3, 3)$$

$$\text{datos_tc3}["x"].\text{reshape}(-1, 3, 3, 1) = \begin{bmatrix} [1.0], [1.0], [-1.0] \\ [1.0], [-1.0], [-1.0] \\ [-1.0], [1.0], [1.0] \end{bmatrix}, \begin{bmatrix} [-1.0], [1.0], [1.0] \\ [1.0], [1.0], [1.0] \\ [1.0], [1.0], [-1.0] \end{bmatrix} \quad (2, 3, 3, 1)$$

Laboratorio: Interpretar la red

Una red puede acertar en una predicción por razones equivocadas.
Se debe:

1. Estudiar los kernels de la primera capa convolucional

- Para extraer el resumen de la arquitectura del modelo se usa: `model.summary()`
- Para extraer la primera capa: `model.layers[0] = Conv2d_4` (depende del # de veces que se ejecutó la celda del modelo)

→ Sacar los pesos entrenados de esta capa: → Para esto se usa `get_weights()`
 Se debetener en cuenta que hay 2 tipos de pesos: → pesos de los filhas
 → pesos de los sesgos

filho 0

```

i=0
j=0 [[[-0.19775027 0.2229062 -0.20104587 -0.05349398 -0.04233126
        -0.10355709 0.04671261 0.27207232]]] → Posición 1 del Kernel.

i=0
j=1 [[[-0.26210868 -0.15850866 -0.27088615 -0.13297671 0.17707428
        -0.05947012 0.0505892 -0.26850644]]] → Posición 2 del Kernel.

i=2
j=2 [[[-0.16126686 0.1606561 -0.04333471 0.13677207 0.18123046
        -0.19921167 0.1575335 -0.16848181]]]

W_filho0 W_filho1 W_filho2 W_filho3 W_filho4
i=0
j=0 [[[-0.06933635 -0.15978712 -0.15161286 -0.23276435 -0.1416542
        -0.00923336 0.22879076 0.2704851]]] →
        W_filho7

i=1
j=1 [[ 0.0115476 0.09626111 -0.07362989 0.12739217 -0.16887072
        0.09073675 0.00609657 0.18413273]]]

i=1
j=2 [[ 0.19095474 0.00670525 -0.11692359 0.04478174 0.22362727
        -0.14318526 -0.09243779 0.05965757]]]

i=2
j=0 [[ [ 0.05868009 -0.19495033 0.07706842 -0.03666747 -0.13653128
        -0.08609632 -0.19553408 0.07847163]]]

i=2
j=1 [[ 0.15409225 0.01032117 -0.20987396 -0.17697904 0.23263937
        -0.25780475 0.23363268 -0.19103286]]]

i=2
j=2 [[ [ 0.05943066 0.17750713 -0.01261055 0.11332318 -0.21006799
        -0.11931553 0.13292283 -0.22426084]]]] → Posición 9 del Kernel.
  
```

(3, 3, 1, 8)

Kernel:

pos1 $w_0 - w_7$	pos2 $w_0 - w_1$	pos3 $w_0 - w_7$
pos4 $w_0 - w_7$	pos5 $w_0 - w_7$	pos6 $w_0 - w_7$
pos7 $w_0 - w_7$	pos8 $w_0 - w_7$	pos9 $w_0 - w_7$

Para armar el `filho 0` completo, necesito
 tomar el primer número de cada una de esas
 9 posiciones

→ Se establece `val_max` como el valor absoluto más grande entre los 8 filtros

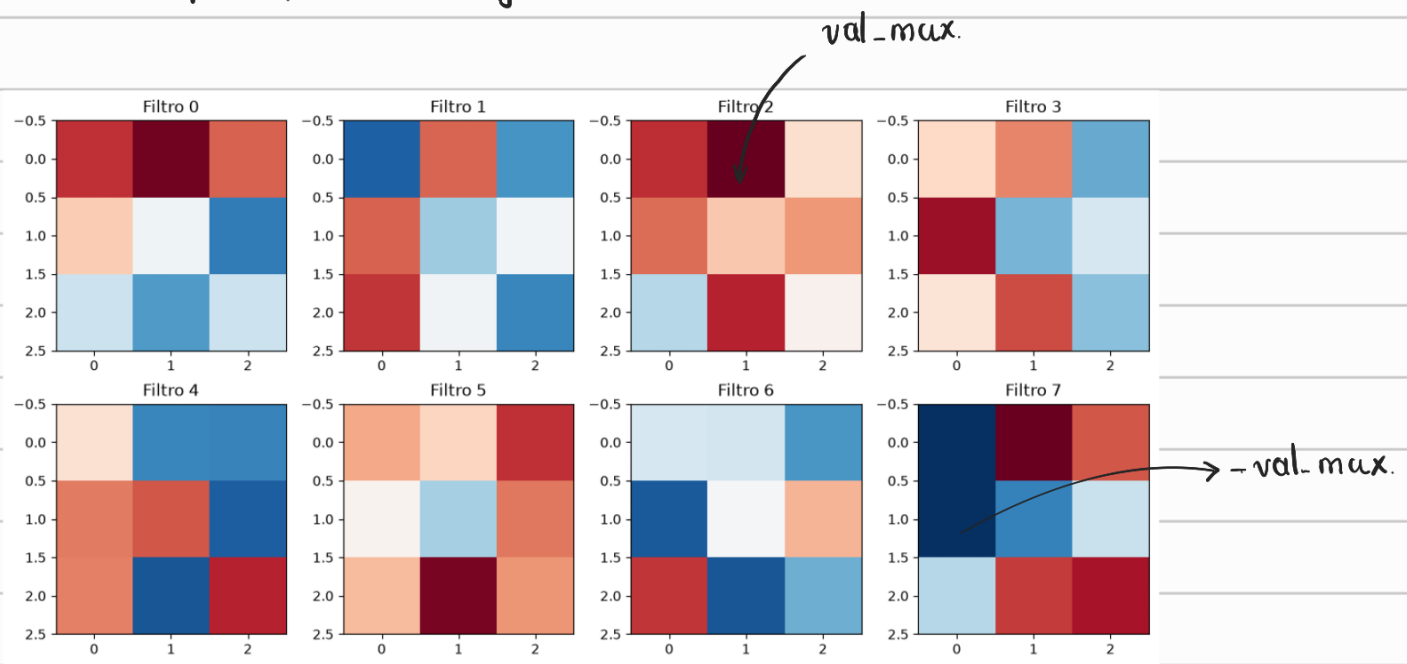
`cmap = "RdBu"` : Rojo, blanco, azul
se centra el blanco en 0.

→ Rojo : peso positivo → mayor valor de peso → rojo más oscuro

→ Azul : peso negativo → mayor valor negativo en magnitud → azul más oscuro

→ Blanco : peso ≈ 0 → casi sin peso

Son los 9 números que aprendió cada filtro



Lo que el peso dice: Se va a amplificar con fuerza (y en la misma dirección)
lo que sea que encuentre en esa posición". Mientras que
un peso cercano a cero significa que "no importa mucho
lo que hay ahí", casi no influye en la salida.

La respuesta a los demás ejercicios están en el notebook de la actividad